

DreamGraph Easy Start

Start here. You do not need to understand the internals before using DreamGraph. Pick the path that matches your project, run the short command list, and open Architect. Its Config, Schedules and Status tabs keep routine work in one place; Explorer opens separately when needed.

DreamGraph v14 is browser-first. **Architect** is the main working surface. Its Config, Schedules and Status tabs handle setup and health. Explorer gives you a visual map when you need orientation.

Before you start

Install DreamGraph from the repository root:

```
# Windows
scripts/install.ps1 -Force

# Linux / accepted WSL2 test bed
bash scripts/install.sh --force
```

Open a new terminal and verify the install:

```
dg --version
```

You should see DreamGraph CLI v14.0.2.

Pick your setup path

A. Start a completely new project

Use this when the project folder is empty or does not exist yet.

```
mkdir my-app
cd my-app
dg init --name my-app --project . --transport http
dg start my-app --http
dg scan my-app --depth deep
```

DreamGraph creates `plans/` automatically. You do not need to create it by hand.

B. Read an existing project

Run these commands from the existing repository root:

```
cd path/to/existing-project
dg init --name existing-project --project . --transport http
dg start existing-project --http
dg scan existing-project --depth deep
```

The scan reads the project and prepares DreamGraph's project map. It does not rewrite your application code.

C. Read a multi-repo system

Choose one folder as the main project root. This is usually the folder that contains your system-level README, workspace file, or deployment configuration.

```
cd path/to/system-root
dg init --name platform --project . --transport http
dg attach . --instance platform --repo api=../platform-api
dg start platform --http
dg scan platform --depth deep
```

For additional repositories, open **Architect** → **Config** → **Repositories**, add the names and paths, and save. The configuration page reports whether a setting applies to new work immediately or needs a restart. Keep the list focused. Add the repositories that belong to the system you want Architect to reason about.

D. Bring an existing Codex or Claude workflow

You do not need to migrate chat history. Keep your repository where it is and initialize DreamGraph against that folder.

For Codex CLI, install and sign in to Codex as usual. Open Architect and choose:

- Adapter: `Codex CLI`

- Model: your preferred Codex model or `auto`

For Claude, use the Anthropic API route. In **Architect** → **Config** → **Models**, choose Anthropic and save the API key. In **Config** → **Architect**, choose the API engine, Anthropic provider and your Claude model. The page reports the effective route. DreamGraph does not import Claude Code conversations or provide a Claude Code inline CLI adapter.

Open Architect and Explorer

Run:

```
dg status my-app
```

Use the host and port shown by `dg status`. If the port is `8100`, open:

Tab	URL	Use it for
Status / Config / Schedules	Inside Architect	Persistent operational tabs for configuration, scheduling and health.
Explorer	<code>http://127.0.0.1:8100/explorer</code>	Visual orientation, search, relationships, and tensions. Both <code>/explorer</code> and <code>/explorer/</code> work.
Architect	<code>http://127.0.0.1:8100/architect</code>	Your main working tab: ask questions, make plans, implement changes, and verify work.

Use them together simply:

1. Start in **Architect** and describe what you want in normal language.
2. Use **Explorer** when you want to understand how parts of the system connect.
3. Use Architect's **Config**, **Schedules** and **Status** tabs when setup or health needs attention.

Connect Computer Use

The v14.0.2 installer registers the browser host. Open `chrome://extensions` (or `edge://extensions`), enable Developer mode, choose **Load unpacked**, and select the extension folder printed by `dg browser setup` (normally `~/.dreamgraph/bin/browser-extension`).

Keep the browser open and check that the extension popup says **Connected to DreamGraph**; `dg browser status` confirms the host connection.

In **Config** → **Computer Use**, save Allow, Ask every time or Deny. Codex CLI and native API routes use the DreamGraph backend when connected. After updating, restart your instance and reload the extension. See the complete setup and troubleshooting guide, including Windows Save/Open dialog support and fallback limits.

What to ask Architect first

For a new project:

```
Help me define the smallest useful first version of this project. Create a plan and start with the simplest working slice.
```

For an existing project:

```
Read this project and explain the main user-facing parts, the important workflows, and the first risks I should review. Keep it practical.
```

For a large multi-repo system:

```
Map the repositories at a high level. Tell me which repo owns each major responsibility, then identify the first three cross-repo workflows worth reviewing.
```

Daily commands

Most days, these are enough:

```
dg status my-app
dg start my-app --http
dg restart my-app
dg stop my-app
dg scan my-app --depth deep
```

Use `dg scan` after attaching a project, after adding repositories, or after major structural changes.

Questions and troubleshooting

Do I need to create `plans/` manually?

No. DreamGraph creates `<project>/plans/` when you initialize or attach a project. Architect also safeguards plan creation if the folder is missing.

I already use Codex. What changes?

Keep using Codex authentication. In Architect, select the Codex CLI adapter. DreamGraph supplies project context and governed repository tools through the browser workflow.

I already use Claude Code. What changes?

Your repository stays unchanged. Configure the Anthropic API provider for Architect. Claude Code transcript import and a Claude Code inline CLI adapter are not currently supported.

Can I use a local model?

Yes. Architect → Config → Models can point DreamGraph to Ollama or LM Studio. Use the existing LLM setup guide when you want local model details.

Which tab should I leave open?

Leave Architect open. Keep Explorer beside it for orientation. Use Architect's Status and Config tabs for operations.

Architect opens the wrong project

Run:

```
dg status my-app
dg attach path/to/correct-project --instance my-app
dg restart my-app
```

The browser does not open

Confirm the daemon is running in HTTP mode:

```
dg start my-app --http
dg status my-app
```

If `8100` was already in use, DreamGraph may choose another port. Use the port reported by `dg status`.

Explorer does not load

Both `/explorer` and `/explorer/` are supported. Confirm the daemon is running and use the host and port reported by `dg status`.

I added more repositories but Architect cannot see them

Check the names and paths in Architect → Config → Repositories. Follow any restart notice shown after saving, then scan the attached repositories.

Where do I go next?

Continue with Installation, Your first instance, LLM setup, or Architect only when you need more detail.